

Flow Control × Congestion Control

两类反馈怎样共同限制发送

408 · NET-B04

Mother Interface

$$Usable = \max\{0, \min(rwnd, cwnd) - FlightSize\}$$

rwnd 是接收端容量声明，cwnd 是发送端对路径承载能力的控制状态；共同作用点是尚未确认的在途字节，但反馈源、保护对象与恢复事件不同。

Owners 与停止边界

NET06 Owns receive buffer、advertised window 与 TCP 连接状态；NET07 Owns congestion evidence 和 cwnd 算法。本册只 Owns 两个回路的合成、辨识和同时变化时的推演。可靠字节区间交给 NET-B03，BDP 利用率交给 NET-B05。

1 两个回路必须各画一个 Owner

接收端根据 buffer capacity、已缓存未被应用读取的数据计算可接收空间，并在 ACK 的 Window 字段中通告 rwnd。应用读取释放空间，数据到达消耗空间。路径不会替接收端生成这个值。

发送端根据 ACK 到达、loss/ECN/timeout 等路径证据维护 cwnd。接收端即使 buffer 很大，也不能授权发送端越过自身拥塞窗口。因而：

Receiver loop protects endpoint

Network loop protects path/queues.

2 合成公式中的三个量不能漏

$\min(rwnd, cwnd)$ 是允许在途上限，不是“此刻还能新发多少”。若 $rwnd = 12\text{ KB}$ 、 $cwnd = 8\text{ KB}$ 、已有 6 KB 未确认，则最多再发 2 KB，而不是 8 KB。ACK 推进会减小 FlightSize 并释放发送空间；窗口值变化会改变上限，二者是不同事件。

接收端通告 zero window 时，正常新数据停止，但 persist/zero-window probe 用于恢复丢失的 window update；这不是拥塞超时。反之，cwnd 因 loss 缩小时，不能据此推出 receiver buffer 已满。

3 四种组合状态

更紧约束	可观察证据	正确动作/解释
$rwnd < cwnd$	advertised window 小、buffer/读取事件	受 receiver 限制；等待消费/窗口更新
$cwnd < rwnd$	拥塞算法状态更小	受 path 限制；按 NET07 响应反馈
$rwnd = cwnd$	数值相等	不能仅凭相等判断成因，仍保留两个 Owner
任一为 0	zero receive window 或拥塞状态	必须看是哪一状态为 0，恢复机制不同

窗口扩大也不必立刻产生发送：还需要应用有待发数据、连接状态允许、FlightSize 留有空间。窗口缩小不自动撤回已经在途的数据；做题应按题设状态机处理，而不是把新上限追溯应用于已发送事实。

4 事件账本与调用协议

1. 记录 $rwnd, cwnd, FlightSize$ 与待发数据;
2. 标注事件来源: receiver ACK/window、path ACK/loss/ECN/timer 或 application read/write;
3. 只更新该 Owner 的状态, 再计算新的 min 上限;
4. 减去 FlightSize 得 usable; 不足一个题设发送单位时停止新发;
5. 若 zero window, 进入 probe 分支; 若 congestion signal, 进入 NET07 算法分支。

高频 First Divergence

直接写 $W = rwnd + cwnd$, 是把两个上限误作可叠加资源; 只算 min 不减 FlightSize, 是把总许可当新增许可; 窗口变小就断言丢包, 是忽略 receiver feedback; 收到 ACK 就只增 cwnd, 是漏掉 ACK 同时可能推进可靠性与携带新 rwnd, 但各状态仍由不同规则更新。

Compression

谁容量不足? 证据来自 receiver 还是 path? 哪个状态改变? 已有多少在途? 合成后还能新发多少? 回答完即分别回到 NET06 或 NET07。